Optimizing Autonomous Fleet Logistics via Proximal Policy Optimization

Aksel Kretsinger-Walters, Alena Chan, Andrey Aksyutkin

December 2025

Abstract

This report details the application of Reinforcement Learning (RL) to the complex logistics of autonomous robotaxi fleets. We model the city environment as a dynamic graph and utilize Proximal Policy Optimization (PPO) to maximize fleet profitability and demand satisfaction. By encoding the city as a network of connected nodes—representing vehicle availability and demand hotspots—we demonstrate that PPO with clipped updates generates stable and profitable policies compared to baseline heuristics. We further analyze the impact of high-reward "red nodes" and their adjacent "orange nodes" on agent behavior within our simulation.

1 Introduction

One of the most promising and revolutionary applications of reinforcement learning (RL) is in the domain of autonomous logistics, specifically self-driving car fleets. The challenges facing the autonomous vehicle community span multiple dimensions, including intellectual, ethical, financial, and technical hurdles. In this project, we narrow our focus to optimizing the operational profitability of robo-taxi fleets.

The relevance of this domain is underscored by its enormous commercial and consumer potential. Commercially, applications extend to freight, last-mile delivery, and industrial logistics. On the consumer side, the technology enables robotaxis, improved public transportation, and critical response systems that enhance safety and accessibility. Effectively managing such fleets solves key problems in inventory management and urban planning.

However, monitoring, maintaining, and optimizing a large fleet of self-driving cars is a non-trivial problem. Operators must maximize availability, minimize downtime, and ensure network resilience. Furthermore, they must accurately predict supply and demand to facilitate dynamic pricing and maximize utilization efficiency. While tasks such as scheduling maintenance, recharging vehicles, and minimizing wait times are difficult individually, jointly optimizing across all these dimensions while adapting to distribution shifts is significantly more challenging. This interactive, multi-objective nature makes the problem a natural candidate for reinforcement learning and agentic approaches.

2 Methodology

2.1 Environment and Simulation

We modeled the city as a graph of nodes representing pickup and drop-off locations, with edges representing the roads connecting them. Leveraging the regular structure of cities like New York, we utilized a grid representation where road distances are estimated using Manhattan distance. To

limit training time—since the state-action space scales linearly with grid size—we initially evaluated basic policies on a simplified 4×5 graph before scaling to larger simulations.

Gymnasium Framework We built our simulation environment using the OpenAI Gymnasium scaffolding. Gymnasium abstracts away many implementation details, allowing us to focus on defining the state space, action space, transition model, and reward structures as follows.

OSMnx Integration For the Manhattan simulation, we utilized OSMnx to extract a realistic road network graph from OpenStreetMap data. This allowed us to model the city's irregular layout and road connectivity accurately, providing a more challenging and realistic environment for our RL agent.

State Space The state space was deliberately abstracted away from the underlying graph. This decision was made to facilitate generalization to unseen cities with different layouts. The state provided to the agent included:

- **Globals (Vector)** These global context variables provide the model with temporal and environmental information, allowing it to learn the latent patterns in demand fluctuations
    1. Time of day (normalized to the unit circle)
    2. Day of week (normalized to the unit circle)
    3. Weather state (categorical variable representing weather conditions)
- **Supply-Demand Conditions (Vector)** These variables inform the agent about the current state of supply and demand in the environment
    1. The current ratio of available cars to ride requests awaiting dispatch (requests that had accepted the price provided to them from the model in the step prior)
    2. Vehicles per nodes
    3. Mean lambda value per node (sampled each step from a Poisson distribution with parameter λ)
- **Vehicles (Matrix)** A matrix representation of the vehicles in the environment
    1. Location (X coordinate) - Normalized to [-1, 1]
    2. Location (Y coordinate) - Normalized to [-1, 1]
    3. Battery level - Normalized to [0, 1]
    4. Status: Idle, Assigned, In-Transit, Charging - One-hot encoded
- **Pending Requests (Matrix)** A matrix representation of the pending requests in the environment
    1. Pickup Location (X coordinate) - Normalized to [-1, 1]
    2. Pickup Location (Y coordinate) - Normalized to [-1, 1]
    3. Drop-off Location (X coordinate) - Normalized to [-1, 1]
    4. Drop-off Location (Y coordinate) - Normalized to [-1, 1]
    5. Cust bias (coefficient representing customers' base willingness to pay)

6. Cust temperature (coefficient representing customers' sensitivity to price changes)
7. Estimated Cost (linear product of distance and a per-unit distance cost)
8. Max Wait Time (in time steps) - Maximum time a customer is willing to wait for pickup once they've accepted the price
9. Wait Time (in time steps) - Current time the customer has been waiting for pickup
10. Status: Expired, Price Rejected, Awaiting Price, Accepted Price (awaiting pickup), Assigned (vehicle en route), In-Transit - One-hot encoded

- **Active Rides (Matrix)**

1. Pickups & Drop off location, Estimated Cost - (duplicated from matched request)
2. Price - Price agreed
3. Total Distance - Total distance of the ride
4. Distance Traveled - Distance traveled so far

- **Dispatch Mask (Vector)** A binary vector indicating which vehicles are eligible for dispatching to new requests (1 if eligible, 0 otherwise)
- **Pricing Mask (Vector)** A binary vector indicating which requests are eligible for pricing (1 if eligible, 0 otherwise)

Action Space The actions available to the agent at each time step included

- **Prices:** For each pending request, the price to offer the customer
- **Repositioning:** For each vehicle, the desired (x, y) location to move towards
- **Dispatch Decisions:** For each request, a one-hot encoded vector indicating which vehicle to dispatch (or none)

Transition Model The environment's transition dynamics were modeled to reflect realistic interactions:

- **Action Application:**

1. **Pricing Action:** Calculate the z-score of the probability of acceptance based on price, supply demand imbalance, weather, estimated cost, cusomter bias & temperature.
2. **Reposition Action:** Move **idle** vehicles towards their target locations.
3. **Dispatch Action:** Assign vehicles to requests based on the dispatch decisions made by the agent.

- **State Management:**

1. **Spawn New Requests:** New ride requests were generated based on a Poisson process. Each node in the graph has an associated λ parameter, dictating the average rate of request arrivals. This parameter could vary over time to simulate demand fluctuations.
2. **Update Existing Requests:** The wait time for each pending request was incremented; requests that exceeded their maximum wait time were marked as expired and removed from the pending list.

3. **Update Rides:** Move active rides towards drop-off locations (deterministic, Djikstra); complete rides when the destination is reached, freeing up vehicles. Match vehicles to requests upon dispatch.
4. **Global Variables**: Update time of day, day of week, and weather conditions as per the simulation clock.

Reward Structure The reward system was designed to allow finetuning, and eventual sparsification of rewards:

- **Dispatches**:
  - Penalties: Multiple dispatches to the same vehicle, dispatching an unavailable vehicle, distance from vehicle to pickup location (euclidean distance for simplification)
  - Rewards: None (implicit reward through successful pickups)
- **Requests**:
  - Penalties: Expired requests, rejected prices
  - Rewards: None (implicit reward through successful pickups)
- **Rides**:
  - Penalties: cost per km traveled (fuel, time)
  - Rewards: fare collected per meter traveled, extra reward upon completion

2.2 Proximal Policy Optimization (PPO)

For the fleet distribution algorithm, we utilize Proximal Policy Optimization (PPO) (Schulman et al. 2017). PPO is a variation of the actor-critic architecture that updates the policy using a clipped surrogate objective, denoted as $L_t(\theta)$. This clipping is vital for real-time business decisions, as it prevents large, destabilizing updates that could, for example, erroneously direct the entire fleet to a low-demand area.

Stable updates are achieved by maintaining "old" and "current" policy distributions over the state-action space and limiting the divergence between them. The objective function we maximize via gradient ascent is:

$$L_t(\theta) = \min\left(r_t(\theta)\,\hat{A}_t,\ \text{clip}\big(r_t(\theta),\,1-\epsilon,\,1+\epsilon\big)\,\hat{A}_t\right)$$

where ϵ bounds the divergence (typically $\epsilon \in [0.1, 0.3]$), $r_t(\theta)$ is the probability ratio of the action under the current vs. old policy, and $\hat{A}_t$ is the estimated advantage. We found $\epsilon = 0.2$ provided the most stable and accurate policy.

Our policy model is a 2-layer neural network with a hidden size of 128 and ReLU activation, optimized using Adam. The critic (Value Function) shares this architecture but outputs a single value (advantage), optimized via mean squared error.

3 Results

We compared different policies on a 20-node city grid before scaling up to Manhattan (graph sparsified to 782 nodes, from 4k+).

20-node results To test robustness, we simulated a high-demand center (controlled by parameter λ) moving in a circular trajectory around the city, the position of which was predictable via the weather state provided to the agent. This assumption is realistic, as autonomous fleets (e.g., Waymo) have access to real-time weather and event forecasting.

Evaluations were conducted over 20-minute episodes, reflecting the average ride time and the threshold at which customers typically cancel if a vehicle has not arrived.

Manhattan results After validating our approach on the 20-node grid, we scaled our environment to a realistic Manhattan layout. The complexity of the environment increased significantly, as did the action and observation space. We weren't able to achieve measurable improvements over our heuristic agent baselines.

3.1 Policy Gradient vs. Baseline

20-node Baseline Comparison As a baseline, we used a **NEAREST** car assignment heuristic, where the closest available car is dispatched to a request. If no demand exists, spare cars move randomly.

We observed that the Policy Gradient approach, which proactively relocates cars and intelligently passes on suboptimal immediate demand for greater long-term rewards, significantly outperformed the baseline.

- **NEAREST Baseline Profit:** 7,990.71 units
- **Policy Gradient Profit:** 12,829.93 units
- **Improvement:** $\approx 60\%$

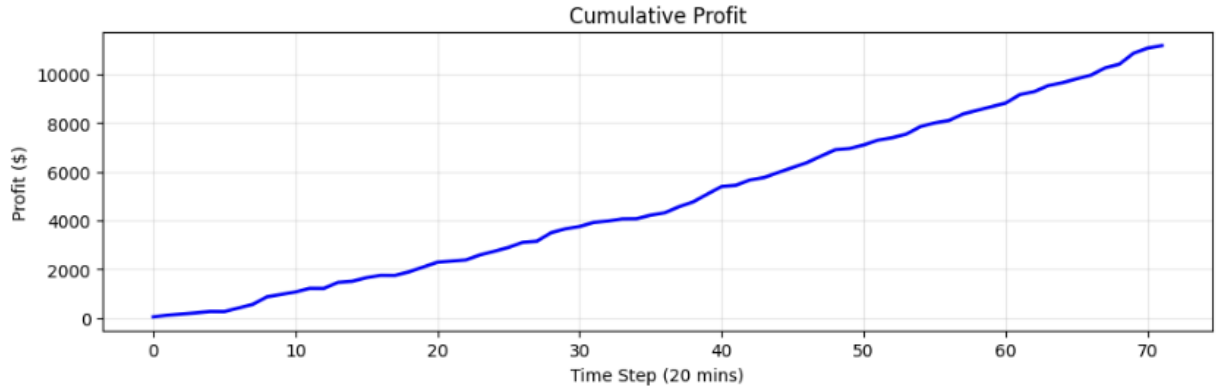

Figure 1: Cumulative profit over a 20-minute episode comparing Policy Gradient to Baseline.

Manhattan Baseline Comparison For the Manhattan environment, we defined the heuristic agents as follows:

- **Pricing Agent:** Price = $(2 \cdot$ Estimated Cost$)$ − Supply/Demand Imbalance (Normalized to [0, 1], higher value indicates oversupply)

- **Dispatch Agent:** Iterate through requests, calculate closest vehicle, dispatch if within threshold distance; otherwise skip.
- **Repositioning Agent:** Assign vehicles to nodes proportionally to the node's degree centrality in the graph.

TODO - explanation? Figures? debugging? next steps?

3.2 Policy Gradient vs. PPO (Clipped)

We further compared vanilla Vanilla Policy Gradient (VPG) against PPO with clipped updates on the 20-node grid with circular demand movement.

While PPO with clipping requires longer training times (more iterations) to learn because it bounds aggressive policy updates, it ultimately achieves higher cumulative rewards and improved stability. This stability is crucial for complex environments where VPG might collapse into suboptimal deterministic policies. Our observations align with theoretical findings in Wang, Zhang, and Gao 2025, which suggest similar long-term performance between clipped PPO and repeated VPG, though PPO offers safety guarantees essential for physical deployment.

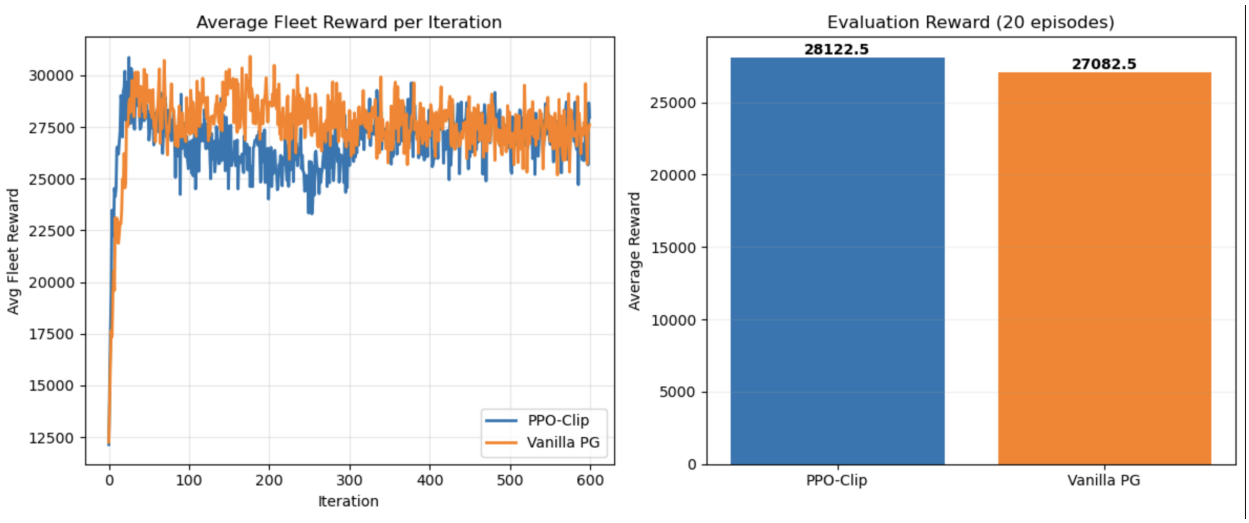

Figure 2: Training curves: Gradient policy update vs. PPO with clipping.

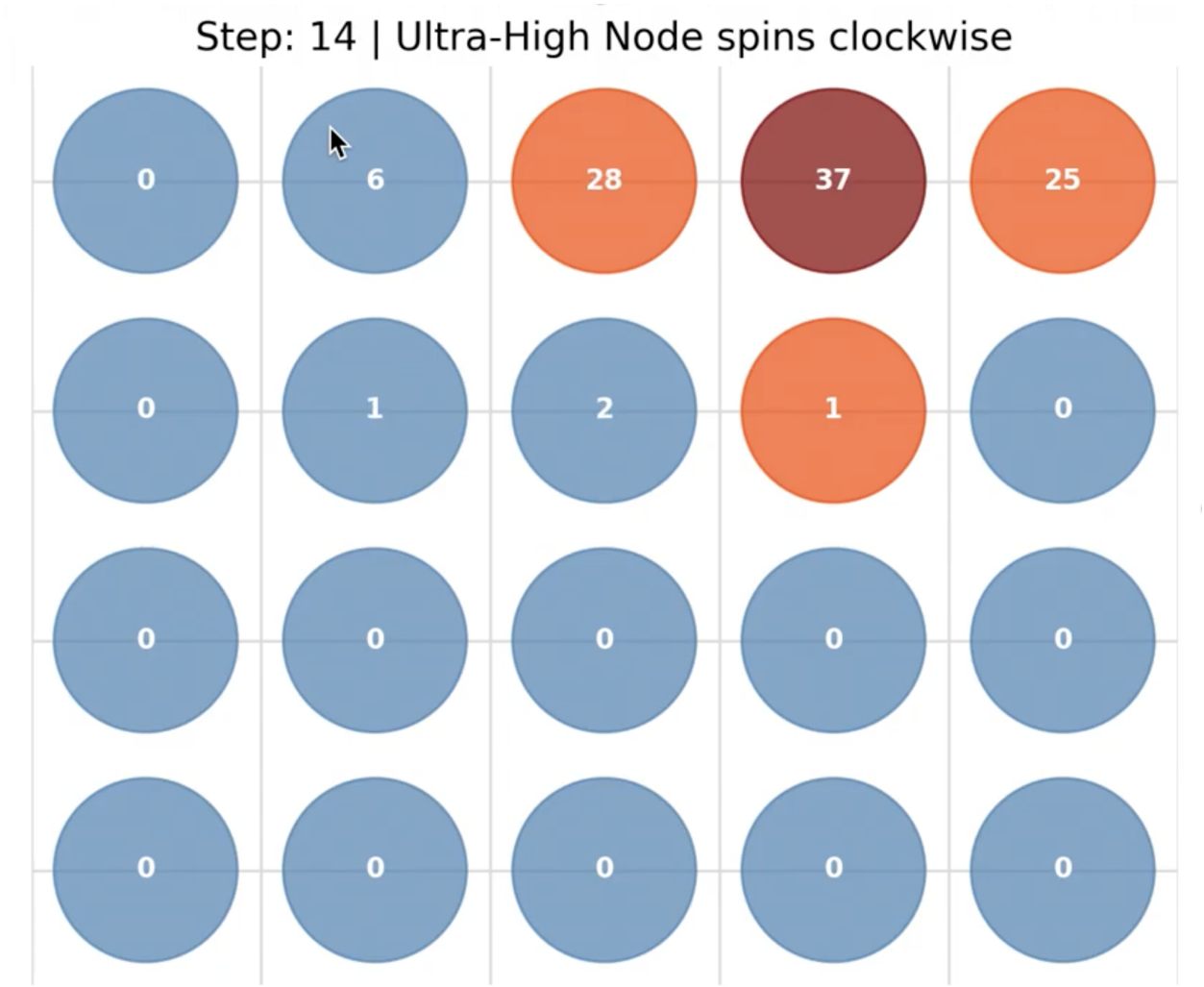

Figure 3: Visualization of trained behavior: Cars aggregating near high-value Red/Orange nodes.

4 Conclusion

In this project, we successfully formulated the autonomous fleet management problem as an MDP and solved it using PPO. By explicitly modeling our city as a simplified graph of nodes containing vehicle count data and highlighting high-reward zones (Red/Orange nodes), our agent learned to anticipate demand rather than merely reacting to it. However, our approach struggled to scale to a larger, more complicated environment.

The code is available at `https://github.com/akseldkw07/Waymo-Agent.git`.
A demo video is available at `https://youtu.be/IXpzuxEX3mU`.

5 Future Work

- **Neural Network Improvements:** Simplification of the observation space, partitioning the agent into specialized sub-agents (pricing, dispatching, repositioning), bootstrapping from

pre-trained models or heuristic agent action, observation, and reward data.

- **Reward Sparsification:** Gradually reducing the frequency of intermediate rewards to encourage long-term planning.
- **Generalizability:** Transfering learned policies to unseen cities with different layouts and demand patterns.
- **Graph Neural Networks:** Utilizing GNNs to better capture the relational structure of the city graph, and remove the intermediate (x, y) coordinate representations.

Our solution could be extended to other cities with irregular layouts. This would require reinterpreting the graph structure, as the Manhattan distance heuristic applies primarily to grid-like cities. Future work would involve integrating OpenStreetMap data (OSMnx) to calculate realistic travel times on non-grid road networks.

References

Schulman, John et al. (2017). *Proximal Policy Optimization Algorithms*. arXiv preprint. DOI: 10.48550/arXiv.1707.06347. arXiv: 1707.06347 [cs.LG]. URL: https://arxiv.org/abs/1707.06347.

Wang, Tao, Ruipeng Zhang, and Sicun Gao (2025). "Improving Value Estimation Critically Enhances Vanilla Policy Gradient". In: *arXiv preprint arXiv:2505.19247*. arXiv: 2505.19247. URL: https://arxiv.org/abs/2505.19247.